⚠ **部分过期 —— 2026-09-04 逐标识符核对**
本页 12 个代码标识符在 **src/ 里已查无此名**，全部属于已从仓里移除的 **DuckLake 层级投影 / locator 发布**支路：
current_hierarchy、current_release、release_id、hierarchy_release(_id)、baseline_hierarchy_release、hierarchy_node、locator_hash、finalize_baseline_with_hierarchy、finalize_attempt_with_hierarchy、DuckLake。
「触发 → 解析 → 模型生成」这条主干仍然成立；凡是提到 DuckLake / hierarchy release 的框一律**按历史资料读，不要照着改代码。**

aios-database 数据解析总流程：从触发到模型生成

HTTP
POST /api/v1/update/execute

文件监听
init_watcher / async_watch

CLI / bin
initialize_project_dbnum_baseline

扫描候选 FileCandidate
（file_latest_sesno = 扫描观察值）
→ 入队 batch_queue（合并 / 冻结区间 / FIFO）

batch_worker（全局唯一消费者）
execute_one_dbnum(project, cand)

applied_sesno = 应用水位（权威值）
「我真的处理完并收口到这里了」

file_latest_sesno = 扫描观察值
「我刚看了一眼，文件长到这」
两者混用 = 把没处理的当处理过了

needs_initial_load ?
applied_sesno == 0
且 file_latest_sesno > 0

否（已有基线）

增量窗口分支
开区间 (applied_sesno, file_latest_sesno]
IncrementPipeline::apply(ranges)

finalize_attempt_with_hierarchy
（与基线同一收口语义）

是（首次建基线）

基线分支 initialize_dbnum_baseline
读 applied_sesno、pe_count、dbnum_info 计数
查 DuckLake current_release(dbnum)

baseline_needs_full_parse ?
pe_count == 0 或 applied_sesno == 0
或 DuckLake release.end_sesno ≠ file_latest_sesno

否（跳过解析）

是

解析本身幂等（replace_dbs=false）
所以「白跑一遍」只损失时间，
不损失正确性。

但不要手动开 total_sync：
sync_pdms 不写水位，独立跑完
基线路径还会再解析一遍。

【解析核心】sync_total_async_threaded
受限选项 baseline_sync_options:
total_sync=true / replace_dbs=false
included_db_files=[本文件] / gen_model=false
详细展开见第 2 页

流式并发写

SurrealDB
pe / pe_owner / ATT_* / ATT_UDA

攒齐后一次性发布

DuckLake
hierarchy_node / hierarchy_release
每次发布 = 一个 snapshot

baseline_model_plan（仅 DESI）
全库活跃 PE → 归一到生成根 → 去重 → RegenRoot
CATA / SYS meta 返回空计划

baseline_hierarchy_release
从 DuckLake 读回 release_id / locator_hash
再校一次 end_sesno == file_latest_sesno
对不上就不推进水位

必须先收口再生成：
生成依赖 current_hierarchy 的当前态
查询，而它要等 release 写进水位后
readiness 门才放行。

数据成功、模型失败不回滚数据。

finalize_baseline_with_hierarchy（单事务）
写 pending 工作项 + 推进 applied_sesno
+ 写 hierarchy_release_id / locator_hash

SurrealDB
dbnum_watermark
model_update_pending

kv-mem readiness 门
applied_sesno 与 hierarchy_release_id
是否同时对上当前 release ?

未就绪

fail-closed
hierarchy dbnum is not ready
当前态查询直接失败，不回退旧持久路径
（回退会让生成观察到混合 release）

放行

分片冷加载
per-dbnum 进程内嵌 SurrealDB 实例

就绪

drain_non_regen（位姿 / 删除 / 级联）
→ load_pending_model_units_for_retry(dbnum)
→ run_unit_worklist

几何生成
prim_model / loop_model / cata_model / bran_hanger
→ OCCT 造型 → 网格

⚠ **本页整体已过期 —— 2026-09-04 逐标识符核对：42 个代码标识符在 `src/` 里已查无此名**
画的是 **`parse_pdms_db` 时代**的基线全量解析路，那条路已经整体退役（退役收口方案 B8.2-B9）。
· **写口**：`save_pes` / `save_pe_relates` 随 `src/versioned_db/pe.rs` 整个文件删除 → 今天由 `versioned_db/baseline_rs::render_baseline_batches` 撅批，行形状取自 `interface/ele_ops.rs::ElementRows`（**基线与增量共用同一份**，宪法 II）。
· **解析核心**：`parse_file_db` / `basic_data` / `refno_table_map` / `parse_file_map` / `type_ele_map` / `ses_range_map` 全部删除 → 今天走 `frozen_source::FrozenSource::capture + e3d_collector::stream_baseline`（e3d-io **退化窗口**，按目标会话索引分块枚举，块交出去就释放，峰值内存随块不随库）。
· **整条 `DuckLake / 层级投影支路`**（`HierarchyRowV1`、`BaselineProjection`、门 A / B / C、`hierarchy_rows`、`sibling_orders`、`locator`、`DuckLake` 本身已从仓里移除——本图与它的三张导出图是**全仓最后一处还在描述它**的地方。
· **`其余一律按历史资料读，不要照着改代码`**。现行口径见 ADR-058 的 2026-09-04 实施注记与 `docs/plans/2026-09-04-legacy-parser-retirement-endgame-plan.md`。
下面只有标了「今天」的两个框是现行的，

解析核心：一次解析，两条互不相干的写路径（历史形态）

本页是第 1 页【解析核心】的展开。

基线路径用受限选项调用：
total_sync=true / replace_dbs=false
included_db_files=[本文件]
manual_db_nums=[本 dbnum]
gen_model=false / gen_mesh=false

所以只解析这一个库，且不顺带生成。

共用解析主链

入口 sync_total_async_threaded
(db_option, project, dbno_set, db_types, progress)

遍历 children_files（逐个 db 文件）
读 60 字节文件头 parse_file_basic_info → db_type / db_no
过滤：db_types 命中 + included_db_files
(SYS 元数据不受 included_db_files 约束，ADR-007)

PdmsIO::open → get_latest_sesno
sesno == 0 直接 bail（没有可发布的 session）
io.open() 顺带构建 ses_range_map

parse_file_db_basic_data → db_basic
refno_table_map / children_map / world_refno
解析失败绝不退化成空库，整批 baseline 失败

门 A · 层级预计算（写盘前）
1) is_projected_hierarchy_noun 过滤，挡掉 MNUM 等数据库控制记录
2) children_map → sibling_orders；子节点重复出现 / 父不是元素 → 失败
3) required_hierarchy_refnos = 全部层级子节点 + world_refno
4) hierarchy_ref0s 从 refno_table_map 取（排除 0 与哨兵）

chunk 循环 all_refnos.chunks(sync_chunk_size)
parse_file_with_chunk(db_basic, chunk, ses_range_map, ignore_world_refno)
→ total_attr_map / type_ele_map
(WORL 只在持有它的那一块解析一次)

SurrealDB 生产者

今天（2026-09-04）：ElementRows.owner_edges，500 条一批
→ SenderJsonsData::PERelateJson
→ INSERT RELATION IGNORE INTO pe_owner
此前：save_pe_relates(db_basic)，随 pe.rs 删除

今天（2026-09-04）：baseline::render_baseline_batches(chunk, dbnum, uda_keys, pe_chunk, att_chunk)
→ SenderJsonsData::PEJson / AttJson
→ INSERT IGNORE INTO pe / ATT_*
此前：save_pes(db_basic, total_attr_map, db_no, ...)，随 pe.rs 删除

按 noun 分表逐条生成 JSON
att.gen_sur_json() → 属性表
att.gen_sur_json_uda() → normalize_sql_string → ATT_UDA
→ SenderJsonsData::AttJson((表名, jsons))

PERelateJson
PEJson
AttJson

SurrealDB 写入管线

flume::bounded(BASELINE_QUEUE_CAPACITY)
背压：无界队列会把未落盘的 INSERT
payload 全留住，撑爆进程

写入 worker × BASELINE_WRITE_WORKERS
chunks(BASELINE_WRITE_WINDOW)
execute_surreal_checked + 冲突重试
(乐观事务下无界并发曾导致静默丢写)

SurrealDB
INSERT IGNORE INTO pe
INSERT RELATION INTO pe_owner
INSERT IGNORE INTO {noun} / ATT_UDA

写入侧排干

DuckLake 层级投影

HierarchyRowV1::from_named_attr_map
refno 一致性校验（map vs attr）
validate_member_owner：成员表里的父
与标量 OWNER 不一致 → 直接失败

hierarchy_rows 内存累加（不立即写盘）
只有 6 个字段：dbnum / refno / owner / noun / name / sibling_order

门 B · 完整性（单文件解析结束后）
parsed_refnos 必须覆盖 required_hierarchy_refnos
缺一个就整库失败，错误里带 parsed / required / missing_sample

门 C · BaselineProjection::new_with_ref0s
ref0 为 0 或哨兵 → 失败；ref0 不在文件索引内 → 失败
row.dbnum 不符 / refno 重复 → 失败
validate_hierarchy_rows → 按 refno 排序 → 推入 hierarchy_baselines

解析侧结果

两条写路径彼此独立，不在同一个事务里：
· SurrealDB 边解析边流式并发写
· DuckLake 全部攒在内存，等 SurrealDB
写管线收尾后才一次性发布

崩溃窗口（PE 已落、DuckLake 未提交）
靠水位与 readiness 门兜底，不靠事务。

finish_write_pipeline(sender, insert_handles, parser_outcome)
关闭 sender → drain 全部 writer → 汇总解析错误与写入错误
(解析失败也要先把 writer 排干，再上抛)

spawn_blocking：HierarchyProjectionStore::publish_baseline
幂等短路：end_sesno + source_digest + change_hash 全等 → already_committed
release_id = hash(project, dbnum, base_release_id, 0, end_sesno, digest, change_hash)

prepare_locator + stage_locator
内容寻址 ref0 → dbnum 反查表
先暂存，不激活

DuckLake 事务（DuckDB ducklake 扩展，SHA256 钉版）
BEGIN TRANSACTION
→ DELETE FROM hierarchy_node WHERE dbnum = ?
→ appender_to_catalog_and_db 逐行追加
→ INSERT INTO hierarchy_release(...)
→ CALL set_commit_message('aios', release_id, extra_info => locator_hash)
→ COMMIT（失败则 ROLLBACK）

DuckLake
hierarchy_node（按 dbnum 分区）
hierarchy_release + snapshot

last_committed_snapshot() → snapshot_id
返回 ProjectionCommit{release_id, locator_hash, snapshot_id, row_count}

evict_global_if_resident(project, dbnum)
基线换了 → 立刻丢弃可能常驻的 kv-mem 分片

define_owner_index / define_pe_index
rebuild_dbnum_info_from_pe（仅 total_sync）

返回 parsed_db_infos（dbnum → 元素数）
交回 initialize_dbnum_baseline 做收口（第 1 页）

⚠ **本页大半已过期 —— 2026-09-04 逐标识符核对：26 个代码标识符在 src/ 里已查无此名**
locator 三段发布协议整体不存在了（prepare_locator / stage_locator / bind_baseline_locator_publication / begin_/finish_locator_publication / active_locator / locator_hash / release_id / hierarchy_release 一族），**DuckLake / DuckDB 存储侧**也已从仓里移除。
现行口径：存储职责见 **ADR-058 D1**（Surreal 只存两样——模型面自己的状态与增量摄入的 DESI 属性数据；CATA 不解析进持久层）；水位与就绪门见 **ADR-021**（本页「不写水位就不就绪」的结论仍然成立）。其余框**按历史资料读，不要照着改代码。**

存储职责边界、locator 三段发布协议与就绪门（历史形态）

SurrealDB · 完整元素数据（权威）

pe — 元素主记录
pe_owner — owner 关系边
ATT_{noun} — 按 noun 分表的属性
ATT_UDA — 用户自定义属性
dbnum_info_table — 每库元素计数
dbnum_watermark — applied_sesno
　+ hierarchy_release_id + locator_hash
model_update_pending — 待重试模型任务

写入方式：解析期间流式并发写，
分批事务，一批失败不回滚已写数据
（正确性由水位兜底）

DuckLake · 当前态层级（唯一权威）

hierarchy_node（按 dbnum 分区）
  dbnum / refno / owner / noun
  / name / sibling_order

hierarchy_release
  release_id / base_release_id
  start_sesno / end_sesno
  source_digest / change_hash
  locator_hash / row_count

每次发布 = 一个 DuckLake snapshot，
commit_message 就是 release_id。
DuckDB ducklake 扩展 SHA256 硬编码钉版。

kv-mem · 可丢弃缓存

每个 dbnum 一个独立的进程内嵌
SurrealDB 内存实例，装载该库全部
层级行，建 owner / noun 两个索引。

首次查询冷加载（2000 行一批），
per-dbnum load gate 串行化。

sync 失败或 release 对不上 →
直接驱逐分片，下次查询重新冷加载。

目前无淘汰策略，只增不减；
规格要求的 RSS 观测也未落地。

locator · 内容寻址文件

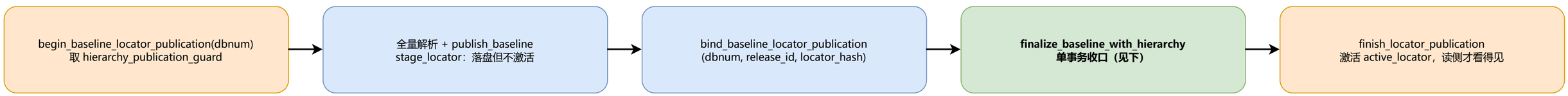
用途：ref0 → dbnum 反查，
以及跨库 owner 路由
（external_owner_routes）。

staged：内容哈希命名，已落盘未激活
active_locator.json：激活指针

发布与激活分离，是为了让
「DuckLake 已提交、水位还没写」
这个中间态不对读侧可见。

每次查找目前都走 spawn_blocking，
整改方案建议换进程内快照。

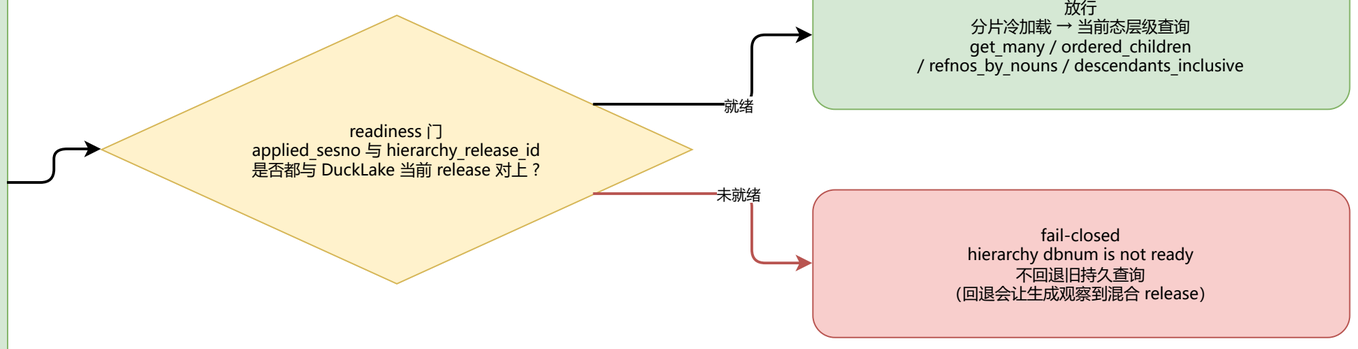
locator 三段发布协议（仅在需要全量解析时启用）



收口事务（唯一推进水位的地方）
render_watermark_advance 只出现在这两个函数里：
finalize_baseline_with_hierarchy / finalize_attempt_with_hierarchy

同一事务内三件事同时成立：
1) 写 model_update_pending 工作项
　(按 (action, target_refno) 去重，靠 revision 收口)
2) UPSERT dbnum_watermark.applied_sesno
3) 写 hierarchy_release_id + locator_hash

任一失败 → 整体不推进，下一轮从头重来。
推进条件是「数据已落库 且 模型工作已持久化」，
不要求模型真的生成完。



几条容易踩的结论
· sync_pdms 发布 DuckLake baseline，但**不写水位**：跑完全量同步系统仍然不「就绪」。
· 更进一步，**独立跑 sync_pdms 基本白费**——applied_sesno 仍为 0，会让 baseline_needs_full_parse 再解析一遍（幂等，只是白花时间）。正确做法是不手动开 total_sync，让基线路径自己驱动那次受限解析。
· SYS meta（SYST / DICT / GLB / GLOB）**也走基线**，不走 cold-start 重放：两条路用的不是同一个解析器，ADR-006 的跨块引用（CURD / DBLS）修复只落在 parse_pdms_db 上。
· 连带后果：UpdateScope 本身是从 SYS meta 库读出来的，所以 SYS 必须先建基线，DESI 才进得了执行范围——batch_worker 的 SCOPE_DIRTY 标志就是处理这件事的。
· readiness 不只是性能负担，它同时是「PE 已落、DuckLake 未提交」这个崩溃窗口的**唯一防线**，优化时只能换粒度、不能拿掉。